

APML Connect Pro

Roles & Responsibilities of Bug Tracking & QA Testing Engineer

TEAM MEMBER GUIDE
Version 2.4 | Quality Assurance

1. Quality Assurance & Testing Role Purpose

In **APML Connect Pro**, the **Bug Tracking & QA Testing Engineer** is responsible for ensuring zero-defect software quality, operational reliability across workstation modules (Reception Desk, Doctor Cabinet, Pharmacy Counter, and Mobile App), network resilience during offline mesh sync ('meshSync'), visual UI perfection, and strict HIPAA security compliance.

The QA Engineer acts as the primary quality gatekeeper before code deployments, systematically detecting, logging, triaging, and verifying bug fixes to guarantee 100% clinical operational uptime.

2. Core Testing Domains & Responsibilities Matrix

End-to-End Clinical Workstation Flow	- Validate complete patient journey: Reception token check-in → Doctor EHR consultation & prescription creation → Pharmacy dispensing checkout. - Test drag-and-drop lab report upload & instant patient search bar.	Reception Desk (3001) Doctor Cabinet (3002) Pharmacy Counter (3003)
Offline Mesh Sync ('meshSync') Resilience	- Conduct network cut/restore tests to verify offline LocalStorage caching. - Validate peer-to-peer data sync without data loss upon server reconnection. - Test vector clock conflict resolution & WebSocket reconnection logic.	WebSocket Gateway Offline Storage Cache P2P Sync Engine
Liquid UI & Visual Design Testing	- Inspect glassmorphism blur cards, neon halos, and dynamic canvas background. - Verify light/dark theme switching, responsive layouts across screens & mobile. - Ensure cross-browser visual consistency (Chrome, Edge, Firefox, Safari).	Liquid Design Tokens Viewport Responsiveness Cross-Browser Suite
Security, RBAC & HIPAA Auditing	- Audit Role-Based Access Control (RBAC): Ensure unauthorized roles cannot access restricted routes. - Verify 2FA/TOTP & SMS OTP authentication flows. - Inspect AuditLog table entries for every patient record read/write event.	RBAC Matrix JWT Auth Middleware AuditLog Verifier
Bug Lifecycle Management & Triage	- Reproduce reported defects with explicit step-by-step reproduction notes. - Classify bugs by Severity (Critical, High, Medium, Low) & Priority. - Capture network payloads, browser console logs, and backend error stack traces.	Issue Tracker / Jira DevTools Console Payload Inspector
API & Backend Regression Testing	- Test Node.js REST API endpoints (Port 5005) for status codes & error handling. - Validate Prisma ORM schema query responses & SQL parameter sanitization. - Run regression suites before production build deployment.	Postman / ThunderClient Jest / Supertest Suite Backend REST API

3. Defect Severity & Classification Standard

Severity Level	Definition & Impact	Required SLA Response
P1 - CRITICAL	System crash, data corruption, workstation outage, mesh sync failure, or HIPAA security breach. Prevents core clinical operations.	Immediate (Fix within 2 Hours) Block release build.
P2 - HIGH	Major functional failure without immediate workaround (e.g., lab uploader crash, prescription saving failure).	Fix within 24 Hours Must fix before release.
P3 - MEDIUM	Minor functional bug with an available workaround (e.g., filter glitch in pharmacy inventory, minor search delays).	Fix within Next Sprint Scheduled sprint item.

P4 - LOW	Cosmetic UI defects, alignment issues, typo in labels, or minor animation stutters.	Backlog Item Fix during UI polish phase.
----------	---	--

4. Standardized Bug Reporting Template & Checklist

When filing a bug report, the QA team member MUST include:

- **Clear Title:** E.g., *[Doctor Cabinet] Patient search fails to render lab PDF when clicked in dark mode.*
- **Environment Specs:** OS (Windows 11/Android), Browser & version, Resolution, Workstation Port (3001/3002/3003/5005).
- **Step-by-Step Reproduction:** Exact numbered actions required to trigger the defect reliably.
- **Expected vs. Actual Result:** Clear statement of what should happen versus what actually occurred.
- **Evidence & Logs:** DevTools Console logs, Network payload screenshots/JSON, and backend terminal error trace.

5. Daily & Release Testing Cadence

1. Smoke test all 3 workstation logins.	1. Run full regression test suite.	1. Re-test target issue in identical environment.
2. Execute manual test cases for new features.	2. Perform offline network disconnection simulation.	2. Conduct side-effect regression check.
3. Log newly discovered bugs.	3. Verify mobile APK installation on Android.	3. Close defect ticket with evidence.
4. Verify developer bug fixes.	4. Sign off on release quality build.	4. Update test case repo.